

Article

Ransomware Detection Model Based on Adaptive Graph Neural Network Learning [†]

Jun Li ^{1,2,*}, Gengyu Yang ^{1,2} and Yanhua Shao ³

¹ Artificial Intelligence Security Innovation Research, Beijing Information Science and Technology University, Beijing 100192, China; yanggengyu@bistu.edu.cn

² School of Information Management, Beijing Information Science and Technology University, Beijing 100192, China

³ National Computer System Engineering Research Institute of China, Beijing 100083, China; stephen_yanhuashao@outlook.com

* Correspondence: lijun@bistu.edu.cn

[†] This article is a revised and expanded version of a paper entitled Real-Time Ransomware Detection Metho-Based on TextGCN, which was presented at 2023 6th International Conference on Artificial Intelligence and Big Data (ICAIBD), Chengdu, China, 26–29 May 2023.

Abstract: Ransomware is a type of malicious software that encrypts or locks user files and demands a high ransom. It has become a major threat to cyberspace security, especially as it continues to be developed and updated at exponential rates. Ransomware detection technology has become a focus of research on information security risk detection methods. However, current ransomware detection techniques have high false positive and false negative rates, and traditional methods ignore global word co-occurrence and correlation information between key node steps in the entire process. This poses a significant challenge for accurately identifying and detecting ransomware. We propose a ransomware detection model based on co-occurrence information adaptive diffusion learning using a Text Graph Convolutional Network (ADC-TextGCN). Specifically, ADC-TextGCN first assign self-weights to word nodes based on sensitive API call functions and preserve co-occurrence information using Point Mutual Information Theory (COIR-PMI); then our model automatically learn the optimal neighborhood through an Adaptive Diffusion Convolution (ADC) strategy, thereby improving the ability to aggregate long-distance node information across layers and enhancing the network's ability to represent ransomware behavior. Experimental results show that our method achieves an accuracy of over 96.6% in ransomware detection, proving its effectiveness and superiority compared to traditional methods based on CNN and RNN in ransomware detection.

Keywords: adaptive diffusion convolution; deep learning; graph convolutional network; network intrusion detection; ransomware detection



Citation: Li, J.; Yang, G.; Shao, Y. Ransomware Detection Model Based on Adaptive Graph Neural Network Learning. *Appl. Sci.* **2024**, *14*, 4579. <https://doi.org/10.3390/app14114579>

Academic Editor: Luis Javier García Villalba

Received: 11 March 2024

Revised: 22 April 2024

Accepted: 24 April 2024

Published: 27 May 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Ransomware is a type of malicious software that encrypts or locks users' files or systems to obtain their private information and demands a high ransom from the users. If the ransom is not paid in time, the ransomware may leak the user's private information. Modern ransomware emerged around 2005 and quickly became a viable business strategy for attackers [1,2]. A notorious example of ransomware is WannaCry, which quickly spread across many computer networks in May 2017 [3,4]. Within a few days, it infected over 200,000 computers in 150 countries [5]. In recent years, ransomware has been growing exponentially, expanding its reach across different platforms, and continuously evolving. This escalation has made ransomware detection crucial not only for protecting individuals' private information but also for improving the overall governance of cyberspace security, thereby making it a primary focus in current information security risk detection research [6,7].

Ransomware can be classified into three main types: crypto, locker, and scareware [8,9], as shown in Figure 1. Crypto ransomware extorts users by encrypting important files stored on their hard drives, making them inaccessible unless a ransom is paid to obtain the decryption key. Locker ransomware locks down the entire operating system, preventing users from accessing their desktop or any personal files and demands a fine to unlock the computer. Unlike crypto ransomware, it usually does not encrypt files but severely limits access to the computer. Scareware falsely informs users that their system has security issues or needs to clean up junk files, often using pop-up windows or warnings to intimidate users into paying for the “full version” of the software to fix these non-existent problems. The evolution of ransomware often involves hackers modifying existing codes to develop new variants, which, while differing in code structure, share similar behavioral goals. According to recent studies, over 98% of ransomware stems from established families, often incorporating or adapting functions from their predecessors [10]. Hackers might also intersperse normal behaviors within the ransomware code to evade detection [11]. The most commonly used ransomware detection methods are static detection and dynamic detection [12]. Static detection focuses on feature extraction and matching of malicious programs in binary form in non-running states. Dynamic detection builds a simulated running environment for specified ransomware samples and records ransomware behavior. Despite their extensive use, these technologies suffer from high rates of false positives and false negatives [13], primarily because they fail to recognize patterns involving non-continuous global word co-occurrences—critical for identifying ransomware.

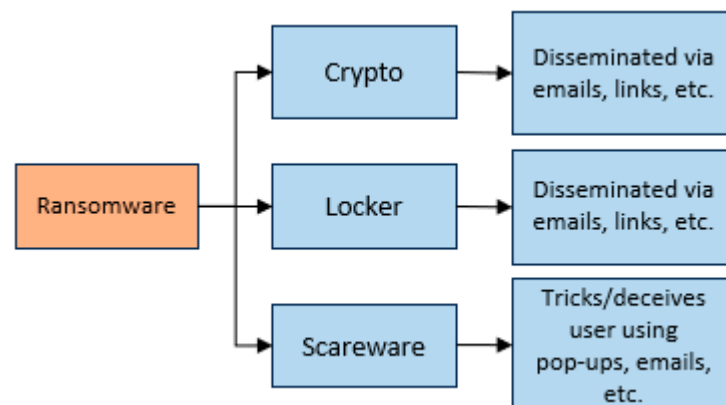


Figure 1. Categories of ransomware.

Recent advancements underscore the importance of innovative technologies in combating the evolving threats posed by ransomware. Apruzzese et al. [14] investigated the use of machine learning and graph neural networks in network security, conducting an in-depth analysis of these technologies in intrusion, malware, and spam detection. This research evaluated the current effectiveness and maturity of these solutions, which are crucial in addressing ransomware’s rapid evolution and complex evasion tactics. Especially, graph neural networks have become a powerful tool for text representation learning, significantly enhancing the understanding of semantic relationships in malicious software code [15].

However, while CNN [16] and RNN [17] are adept at extracting features from continuous word sequences, they often overlook global word co-occurrence and the associations between key node steps. Global word co-occurrence information refers to the relationship between words that appear discontinuously in the text. They carry long-distance semantic information, which is crucial for ransomware recognition. For example, if the words “encryption” and “ransom” appear in the text, even if they are not continuous, it can be judged that this is a feature of ransomware. Consequently, relying solely on local adjacent information may render these methods ineffective, as they struggle to integrate crucial global context.

To address this challenge, the paper introduces ADC-TextGCN, an innovative ransomware detection model. It leverages co-occurrence information and adaptive diffusion learning through a Text Graph Convolutional Network [18]. ADC-TextGCN is a representation learning method of adaptive graph neural network based on key ransom text, which can represent key ransom text as a word token node, where each node represents a word, and each edge represents the relationship between two words. ADC-TextGCN can use graph convolution operations to extract global word co-occurrence and association relationships from word graphs for the precise learning and recognition of ransomware. To achieve the adaptive representation and learning of the different text relationships of ransomware, it is very important to effectively aggregate multi-hop node information. Only when the node representation has enough global relationship knowledge can the whole learning process be more effective and more useful and the training results be more valuable.

Specifically, ADC-TextGCN uses Point Mutual Information Theory (COIR-PMI) to preserve co-occurrence information to calculate the edge weights between word nodes so as to preserve sequential and co-occurrence information as much as possible and improve the performance of ADC-TextGCN. In the model's phase of learning node information, TextGCN typically captures data only from a node and its immediate neighbors. To enrich node representations with broader contextual information during multi-layer propagation, we introduce an Adaptive Diffusion Convolution strategy to the TextGCN framework. This innovation enables the automatic identification of the most informative neighborhood size for each dataset, facilitating the incorporation of information from more distant nodes across layers. Such an approach allows each dataset to have a tailored propagation neighborhood size, consistent across all layers of the graph neural network and feature channels, significantly enhancing the model's ability to aggregate complex ransomware behavior patterns and improve its overall predictive performance.

To verify the effectiveness and superiority of our method, we conducted experiments on public datasets. We compared the ADC-TextGCN model with several baseline methods, including traditional methods based on CNN, RNN and TextGCN. Experimental results show that our method achieves the best performance on all the datasets, reaching a ransomware detection accuracy rate of over 96.6%, outperforming the performance of the benchmark methods. This proves that our method can effectively learn co-occurrence information and improve the recognition detection ability of ransomware.

The structure of this paper is as follows. In Section 2, we introduce the related research published in recent years. Section 3 details the proposed method. The experimental setup and results are discussed in Section 4. Finally, Section 5 provides a comprehensive summary and discussion of the findings.

2. Materials and Methods

In this section, we embark on an in-depth exploration of pivotal research associated with ransomware detection. We commence by presenting a variety of methodologies specifically designed for the detection of ransomware. This is followed by a thorough elucidation of the process of acquiring API call sequences. The section culminates with a summarization of the diverse applications of Graph Neural Networks in the realm of graph analysis problems.

2.1. Ransomware Detection Methods

In the realm of ransomware detection, a blend of deep learning techniques with dynamic or static detection methods has been employed by several researchers. EldeRan [19], for instance, is a dynamic analysis and classification approach that leverages machine learning to combat ransomware. This method scrutinizes a set of operations executed within the initial 30 s of running the analyzed application. It employs Mutual Information (MI) and standard Logistic Regression (LR) to anticipate impending ransomware attacks. When evaluated on a dataset comprising 582 ransomware applications from 11 families and 942 benign applications, EldeRan achieved an Area Under the ROC Curve of 0.995.

Kharraz [20] analyzed samples submitted to UNVEIL, tracked I/O access, and calculated image difference scores before and after execution to monitor system activity for abnormal files.

Presently, there are numerous methods for detecting ransomware. One such method automatically extracts ransomware features from behavior logs generated by various benign and ransomware analysis reports using the Cuckoo Sandbox tool [21]. This tool monitors and categorizes suspicious software activities, such as alterations to files and folders, memory dumps, network traffic, and the handling of API calls. Subsequently, it employs Term Frequency–Inverse Document Frequency (TF-IDF) to automatically identify and rank the most significant ransomware features in the recorded data. Feature frequency can also be utilized for ransomware detection [22]. This method, based on the dynamic analysis of 1412 ransomware and 379 benign software applications from 16 families, calculates the frequency of ransomware behavior for specific suffixes, paths, and API calls and combines it with memory behavior to construct features. A ransomware detection model is then established using a parameter-optimized Random Forest algorithm. Moreover, by monitoring the real-time behavior of program operations and extracting features from API calls and registry operations, supervised learning is utilized to detect ransomware [23]. Similarly, the dynamic detection method using TextCNN treats APIs as text languages, constructs graph convolutional networks, and preserves word order information by enhancing pooling layers [24].

The amalgamation of dynamic and static methods effectively enhances detection accuracy. The multi-classifier ensemble method based on features such as permissions and program intentions [25] and the set detection model based on Random Forests and Decision Trees [26] have effectively improved the system's detection accuracy, yielding satisfactory detection results. Building upon previous research, we propose ADC-TextGCN. This method also integrates dynamic analysis and static analysis for text graph construction to augment detection accuracy.

2.2. Acquisition of API Call Sequences

API call sequences are derived from the behavioral logs within the Cuckoo Sandbox, where both ransomware and benign software are executed. Based on the behavior observed and documented in studies such as the execution process of the WannaCry virus [27,28], the ransomware attack can be broken down into five key steps: establishing network connectivity, generating asymmetric keys for encryption, establishing communication with remote command and control (C&C) hosts, encrypting data on the host system, and finally, releasing ransom information to the victim.

Through the analysis of these steps, sensitive API functions are identified. These sensitive API call functions are then integrated with the dynamic features obtained in the Cuckoo Sandbox to form an API call sequence text graph. During the construction of this text graph, the weight between the word nodes is adjusted using the identified sensitive APIs, thereby amplifying the features related to these APIs. Simultaneously, while calculating the weight between word nodes, the statistical rules of the Pointwise Mutual Information (PMI) theory are modified to preserve both word order information and word co-occurrence information. Finally, the constructed API call sequence text graph is input into the ADC-TextGCN, and the classifier subsequently produces results after discrimination. This methodical approach ensures a comprehensive and effective evaluation of potential ransomware threats.

2.3. Graph Neural Network Correlation Method

In the realm of graph neural networks (GNNs), a comprehensive review [29] offers profound insights into the understanding and effective application of GNNs to address practical problems. The graph convolutional network (GCN) introduces a graph convolution operation, capturing node relationships via an adjacency matrix and learning representations of adjacent nodes and its own nodes through a convolution operation. The seminal work of Kipf and Welling (2016) [30] established the foundation for GCN, with

their innovative ideas and methods significantly influencing subsequent GNN research. GCNs [30,31] extend convolution operations to local graph structures, thereby offering impressive performance for various graph mining tasks [32–34]. The primary function of the graph convolution operation is to aggregate information from nodes directly adjacent to the central node, a process also known as message passing [35]. To propagate information between nodes at a greater distance, multiple neural layers can be stacked to exceed the direct jump of neighbors. To directly gather high-order information, spectral-based GNNs utilize graph spectral characteristics to collect signals from global neighbors [34,36,37].

The TextGCN model [18] constructs a text graph in a corpus based on word co-occurrence and the relationship between words in the text, providing a novel approach to ransomware detection using TextGCN based on API call sequences. The inductive learning framework of GraphSAGE [32] effectively utilizes node attribute information to efficiently generate embeddings for previously unseen nodes. The Graph Attention Network (GAT) [33] introduces an attention mechanism, effectively utilizing node attribute information to efficiently generate embeddings for unknown nodes. Graph Diffusion Convolution [38] aggregates information not only from single-hop neighborhoods but also from larger neighborhoods, thereby capturing long-distance contextual information and better characterizing node features.

The Adaptive Diffusion Convolution strategy [39] is a deep learning method specifically designed for GNNs, marking a significant innovation. It allows each GNN layer and each feature channel to have different neighborhood sizes and can automatically learn the optimal neighborhood size from the data, thereby enhancing the effectiveness and performance of graph convolution and improving the model's adaptability to different graph structures and feature dimensions. Adaptive Diffusion Convolution offers several advantages: (1) It can adaptively select the most suitable neighborhood size based on different graph structures and tasks, eliminating the need for manual adjustment or a grid search. (2) It can learn a dedicated neighborhood size for each GNN layer and feature channel, enabling the GNN to better couple with the graph structure and feature channels, thereby enhancing the model's expressive power. (3) It can be directly inserted into existing GNNs without altering the network's depth or width and without incurring significant computational costs.

A key feature of Adaptive Diffusion Convolution is its ability to automatically learn the optimal neighborhood size from the data. This means it can determine how many neighbors should be considered based on the characteristics of each node and the requirements of the task, allowing Adaptive Diffusion Convolution to better capture the local and global structures of the graph. Adaptive Diffusion Convolution is a method that uses an adaptive diffusion kernel to diffuse images or feature maps. This method allows Adaptive Diffusion Convolution to dynamically adjust the shape and size of its diffusion kernels based on the characteristics of the input data, ensuring that it meets the specific information aggregation needs of different ranges and scales. Consequently, we consider using Adaptive Diffusion Convolution to address the co-occurrence information learning challenges in ransomware detection.

3. Methodology

The main idea of ADC-TextGCN is to learn a dedicated propagation neighborhood for each GNN layer and feature channel so that the GNN architecture is fully coupled with the graph structure. This is a unique characteristic of GNNs that differs from traditional neural networks. The implementation of ADC-TextGCN formalizes the task as a two-layer optimization problem, allowing for the customized learning of an optimal propagation neighborhood size for each dataset. A two-level optimization problem is a special type of optimization problem in which the solution of one optimization problem (usually referred to as a lower-level problem) is constrained to the optimal solution of another optimization problem (usually referred to as an upper-level problem). These two issues interact with each other, forming a hierarchical structure. During the transmission of messages in each

graph, all GNN layers and feature channels (dimensions) share the same neighborhood size. To further develop in this direction, ADC-TextGCN also allows for the automatic learning of custom neighborhood sizes for each GNN layer and each feature channel from the data.

The processing flow framework of the ADC-TextGCN is based on TextGCN, as shown in Figure 2. First, we run both ransomware and benign software in a controlled area called a sandbox. Then, we extract API call sequences from the behavior logs of these programs. Leveraging both static and dynamic analysis methodologies, a text graph of the API call sequence is constructed and fed into the ADC-TextGCN network. The Adaptive Diffusion Convolution strategy is employed across each layer and feature channel within the ADC-TextGCN network, facilitating the adaptive acquisition of the optimal neighborhood size. Ultimately, the outcomes are produced via a classifier model.

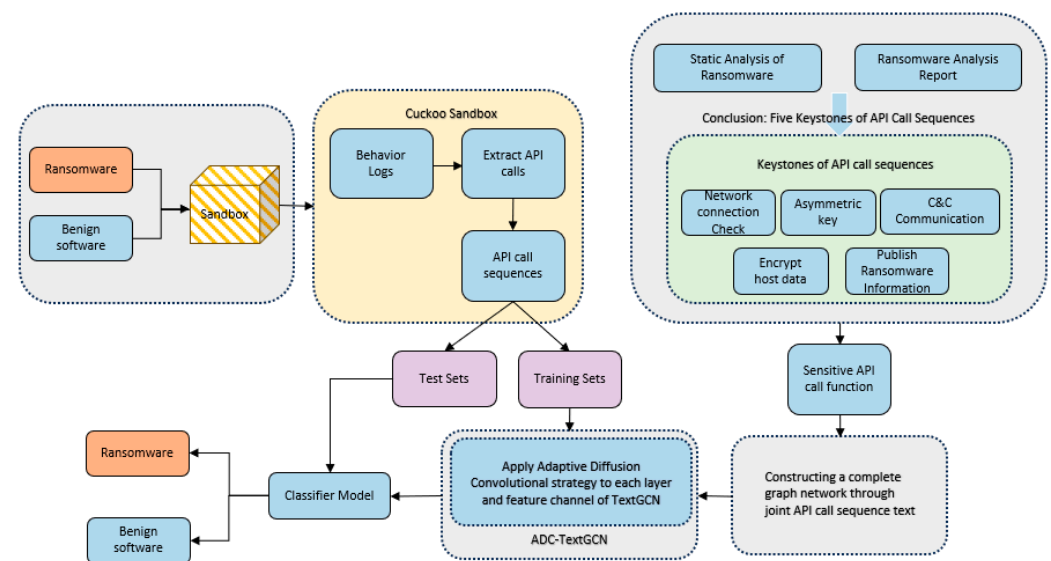


Figure 2. The processing flow framework of ADC-TextGCN detection model.

Figure 3 illustrates the structure of the ADC-TextGCN and demonstrates the use of neighborhood radius (circles on the right) for semantic information integration for classification purposes. The concept of neighborhood radius is employed to describe the local connectivity patterns between nodes, signifying the co-occurrence strength of different words within the documents and their contribution to classification. In the diagram on the right, each word node is encompassed by a neighborhood radius that visualizes the density of its relationships with other words, as well as its impact on the document classes 'Crypto', 'Locker', and 'Scareware'.

The classifier model used in conjunction with the ADC-TextGCN architecture has been carefully designed to utilize the subtle feature representations extracted by the Adaptive Diffusion Convolution strategy. It typically consists of multiple layers, including an input layer in vectorized form using API call sequences, one or more hidden layers applying adaptive diffusion processes, and an output layer that classifies sequences as ransomware or benign software. Each layer is configured with specific hyperparameters, such as the number of neurons and the type of activation function, to optimize the detection performance of the model. The selection of hyperparameters is determined by extensive experimental testing, aiming to achieve a subtle balance between maximizing detection accuracy and maintaining computational efficiency.

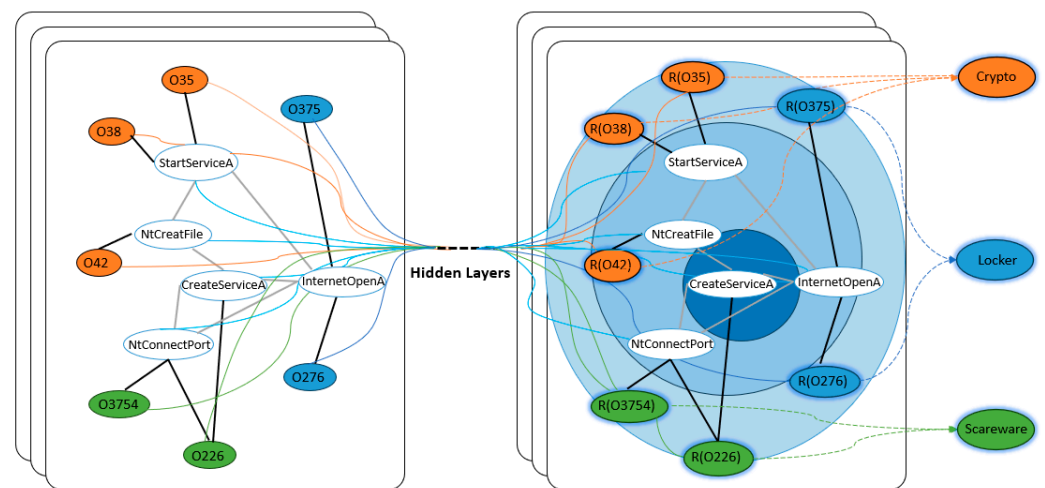


Figure 3. Schematic of the ADC-TextGCN. Nodes starting with ‘O’ are document nodes, while others are word nodes. Black bold edges represent document-word connections, and gray thin edges represent word-word connections. R(x) denotes the representation (embedding) of x. Different colors signify different document categories. The circles on the right-hand side of the figure illustrate the neighborhood radius used to capture and represent node information, with multiple circles emphasizing the concept of adaptability.

The training process of the model is summarized in Algorithm 1.

Algorithm 1: Training process of the ADC-TextGCN

Input: Training set D

Output: Trained ADC-TextGCN network G

- 1: Dataset organization: Organize the training set D .
- 2: Obtain API call sequences: Extract API call sequences based on the training set D .
- 3: Analyze the five key steps of a ransomware attack: By analyzing the five key steps of a ransomware attack, derive the set of sensitive API functions S .
- 4: Construct the text graph: Build a text graph $G_T = (V_T, E_T, W_T)$ based on the API call sequences. For the nodes $v \in V_T$ of the text graph, construct a node weight measurement formula based on the set of sensitive API functions S , and amplify features related to sensitive APIs.
- 5: Input the constructed text graph into the TextGCN network: Input the constructed text graph G_T into the TextGCN network G , integrate the Adaptive Diffusion Convolution strategy, using the neighborhood radius formula to dynamically adjust the diffusion process:

$$r = \frac{\sum_{s=0}^{\infty} \beta_s S}{\sum_{s=0}^{\infty} \beta_s}$$

where β_s represents the influence received from nodes at an s -step distance, allowing the model to adjust its information propagation range based on the node’s connectivity density and structural information.

- 6: Use the classifier model to classify the output: Use a classifier model to classify the output of the ADC-TextGCN network G and generate results.
-

3.1. Extract API Call Sequences and Sensitive Functions

In the field of ransomware detection, the extraction of API call sequences is a key step, and there are mainly two methods: static analysis and dynamic analysis. The extraction of API call sequences involves obtaining the names and order of APIs called by a program from its code or execution process, which can reflect the program’s functionality and behavior, making it suitable for program analysis tasks such as malware detection, code cloning detection, and code search.

The static analysis method extracts API call sequences without executing the program. It analyzes program code from the perspectives of vocabulary, grammar, and semantics.

However, due to the inability to handle dynamically loaded APIs or parse complex control and data flows, it may generate false positives or false negatives.

The dynamic analysis method extracts API call sequences by monitoring, recording, and analyzing the execution process of the program. The dynamic analysis method can accurately obtain the API actually called by the program, without being affected by the code structure, and can handle dynamically loaded APIs. However, it can only cover a portion of program branches and paths and is influenced by input data.

We track the execution process of the program and obtain the API call sequence by running the example executable file in the Cuckoo Sandbox, as shown in Figure 4. In addition, sandbox technology can be used for online virus analysis and malware behavior detection [40].

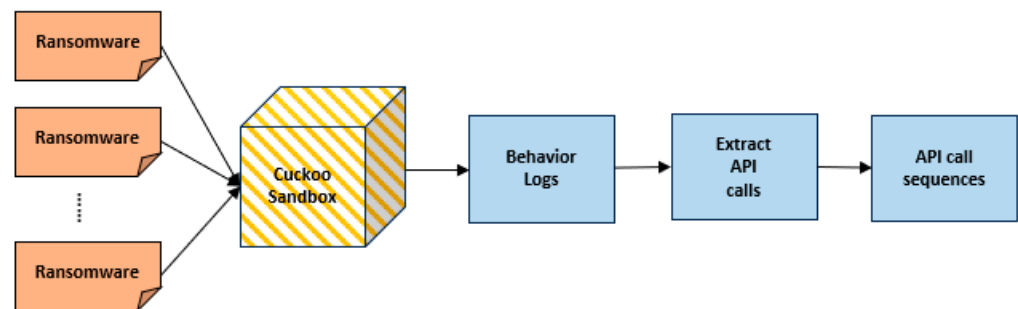


Figure 4. Dynamic extraction process of the API call sequence.

Based on the API-service set, we propose a hybrid API weight evaluation method and map it to graph nodes. We regard sensitive API call functions as sensitive nodes. If node i is a sensitive node, then the weight between node i and its neighbor node j will be increased. This means that the original weight between i and j is multiplied by α , which is a weight growth factor. If both nodes i and j are not sensitive, the original weight between nodes i and j is retained. The value of α is determined by a matrix search.

An example is shown in Figure 5, where the dark-red nodes represent sensitive API call functions, i.e., sensitive nodes. In addition, the light-blue nodes represent general API call functions. The left figure is the original call sequence text graph. Since node 3 is a neighbor node of the sensitive node, the weight between nodes 2 and 3 should be multiplied by the weight growth factor α . However, node 4 is not directly connected to node 2, so A_{1-4} remains unchanged. Node 2 is a sensitive API call function, and nodes 3, 4, and 1 are not sensitive. The weight between nodes 2 and 3 is multiplied by α , but the weight between nodes 1 and 4 is still A_{14} .

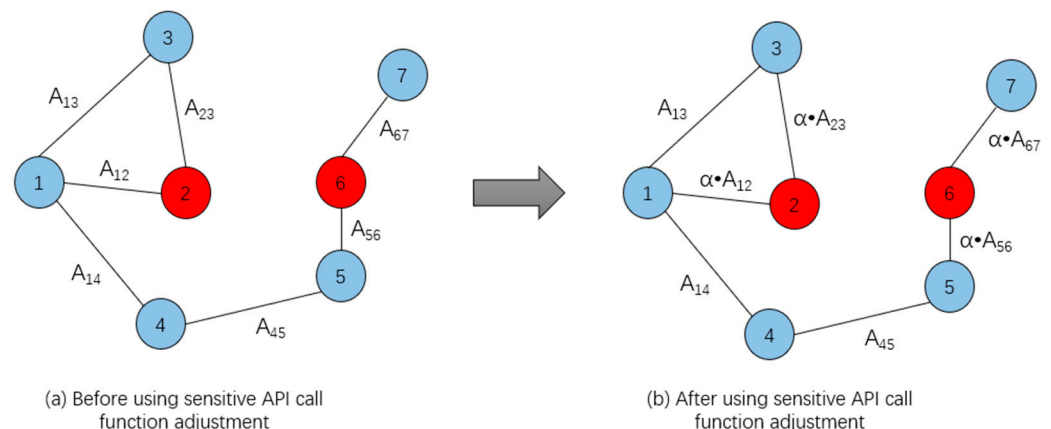


Figure 5. Sensitive API call functions adjust the weights of word nodes. (a) Before using sensitive API call function adjustment, with light-blue nodes representing general API call functions and dark-red nodes indicating sensitive API call functions. (b) after using sensitive API call function adjustment.

Building on this foundation, to understand the relationship between non-adjacent API calls in the sequence, we introduce the Adaptive Diffusion Convolution mechanism. This mechanism evolves from Graph Diffusion Convolution, which uses the manual adjustment of the neighborhood size for feature aggregation into an adaptive learning size. Adaptive Diffusion Convolution can automatically learn the neighborhood radius of each graph neural network (GNN) layer and each feature dimension. Its adaptive mechanism allows each node to have a neighborhood radius suitable for itself, which can be integrated into the learning process of the co-occurrence information of ransomware.

For a sensitive node with a larger weight, a node that is highly similar to it and has co-occurrence information may not be its neighbor node. The Adaptive Diffusion Convolution method can adaptively adjust the neighborhood radius of the node to the optimal situation, and the neighborhood radius will include that node. By aggregating more node co-occurrence information, the center point can be better enriched, potentially leading to improved learning results.

3.2. Constructing Graph Network by Combining API Sequences

A graph is constructed from the API call sequence text, incorporating word nodes, document nodes, and weighted edges. This structure aims to preserve as much word order information and co-occurrence information as possible. The total number of nodes $|V|$ in the text graph is the aggregate of two types of documents (ransomware and benign documents) and the count of unique API call functions. The API call sequence text graph, as implemented in ADC-TextGCN, is a large heterogeneous text graph, thereby enabling it to explicitly model valuable feature information. Within the API call sequence text graph, API call functions serve as word nodes, while ransomware and benign software are represented as document nodes. The edges between nodes are established based on the occurrence of words within the document and the co-occurrence of words within the API call sequence.

Term Frequency-Inverse Document Frequency (TF-IDF) is a widely used weighting technique in information retrieval and data mining, which assesses a word's relevance to a document within a corpus. TF-IDF consists of two parts: *TF* and *IDF*. *TF* (Term Frequency): This represents the frequency of a term in a text. This number is usually normalized (generally, the Term Frequency divided by the total number of words in the article) to prevent it from favoring longer documents (the same word may have a higher Term Frequency in a long document than in a short document, regardless of the importance of the word). The calculation formula for *TF* is

$$TF_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}$$

where $n_{i,j}$ denotes the number of occurrences of term t_i in document d_j , and $TF_{i,j}$ represents the frequency of term t_i in document d_j .

Inverse Document Frequency (*IDF*): This metric represents the general significance of a keyword. If fewer documents contain the term, the *IDF* will be larger, indicating that the term has good category differentiation ability. The *IDF* of a specific word can be obtained by dividing the total number of documents by the number of documents containing the word, and then taking the logarithm of the quotient. The calculation formula for *IDF* is

$$IDF_i = \log \frac{|D|}{1 + |j : t_i \in d_j|}$$

where the denominator is increased by 1 to avoid the situation where the denominator is zero, $|D|$ represents the total number of all documents, and $|j : t_i \in d_j|$ represents the number of documents containing term t_i . Finally, the calculation formula for TF-IDF is

$$TF-IDF = TF \cdot IDF$$

In this way, TF-IDF tends to filter out common words and retain important words, effectively calculating the weight between document nodes and word nodes. Multiplying TF and IDF can obtain the importance weight of a word in a document. Common weighting methods include $TF \times IDF$, $\log(TF + 1) \times IDF$, etc. It is a statistical method used to evaluate the importance of a word to a document. The importance of a word increases in direct proportion to the number of times it appears in a document, but at the same time, its importance decreases inversely with the frequency it appears in the corpus. To utilize global word co-occurrence information, we use a fixed-size sliding window on all documents in the API call function to collect global word co-occurrence information and word order information. By adopting the COIR-PMI theory of co-occurrence information retention, changing the co-occurrence rules of PMI, and obtaining word order information, the weight between two word nodes can be calculated.

While building the API call sequence text graph, numerous API functions are invoked. A sliding window of size N scans each executable file's API call sequence, forming an N-length fragment. For instance, consider an executable program that sequentially calls 12 functions, as demonstrated in Table 1.

Table 1. Example of API call function.

API Call Function Name	Thread Number	Sequence Number	Function Type	Risk Factor
InternetOpenA	2332	47	Network	Low
InternetOpenUrlA	2332	48	Network	Low
OpenSCManagerA	2332	33	System	High
CreateServiceA	2332	49	System	High
StartServiceA	2332	50	System	High
LdrGetProcedureAddress	2332	51	Loader	Medium
NtQueryDirectoryFile	2332	52	File System	Medium
NtRequestWaitReplyPort	2332	37	Port	Low
NtCreateFile	2332	54	File System	Medium
NtQuery	2332	56	System	High
NtQueryValueKey	2332	42	Registry	High
NtConnectPort	2332	58	Port	Low

The fragment for this API call function when $N = 5$ is shown in Figure 6.

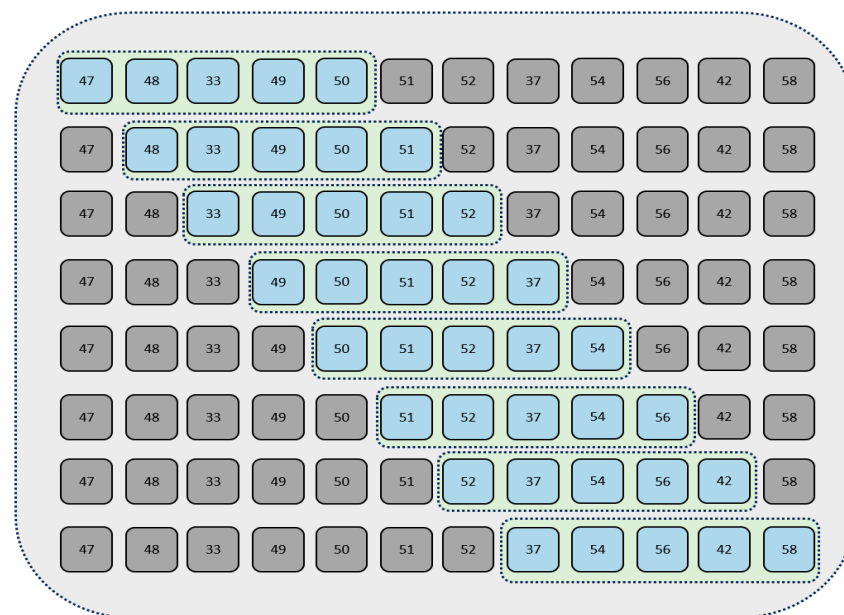


Figure 6. The fragment for the API call function. The green rectangular box represents the sliding window which moves across the API call sequence. Light blue nodes within the window indicate selected API call sequences (Sequence Number), and grey nodes represent those not selected.

In the COIR-PMI method, we have modified the PMI statistical rule to compute the weight of the edge between two word nodes. This algorithm uses PMI to measure the correlation between two variables, preserving both word order and co-occurrence information. We employ a sliding-window approach to extract valid sequence relationships for API call functions. The TF-IDF algorithm is utilized to retain API call interfaces that are pertinent to classification.

Calculating COIR-PMI: The PMI value for word pair i, j is computed as follows:

$$\text{COIR-PMI}(i, j) = \log \frac{p(i, j)}{p(i)p(j)}$$

$$p(i, j) = \frac{\#W(i, j)}{\#W}$$

$$p(i) = \frac{\#W(i)}{\#W}$$

In these equations, $\#W(i)$ represents the count of sliding windows in a corpus that contains word i , $\#W(i, j)$ is the count of sliding windows that contain words i and j in order, and $\#W$ is the total count of sliding windows in the API call sequences.

For instance, consider the calculation of co-occurrence of 47 and 48 in Table 1. The count of co-occurrences of 47 and 48 in the window $\{(47, 48, 33, 49, 50)\}$ is 1, but the count in the window $\{(48, 47, 33, 49, 50)\}$ is 0. Therefore, $\#W(i, j)$ is not equal to $\#W(j, i)$.

$$A_{ij} = \begin{cases} \text{COIR-PMI}(i, j) & (i, j \text{ are words}) \\ \text{TF-IDF}_{ij} & (i \text{ is document, } j \text{ is word}) \\ 1 & (i = j) \\ A_{ij} \cdot \alpha & (i \text{ or } j \text{ is a sensitive API call}) \end{cases}$$

Whether COIR-PMI is positive or negative carries practical significance and represents characteristic information. To retain more comprehensive information, we add edges between word pairs with PMI values.

After constructing the text graph, we feed the graph into a simple two-layer GCN. The second layer node (word/document) embeddings have the same size as the label set and are fed into a softmax classifier:

$$Z = \text{softmax}(\tilde{A}\text{ReLU}(\tilde{A}XW_0)W_1)$$

where $\tilde{A} = D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$, and $\text{softmax}(x_i) = \frac{1}{Z}\exp(x_i)$, with $Z = \sum_i \exp(x_i)$.

3.3. Adaptive Diffusion Convolution of ADC-TextGCN

The Graph Diffusion Convolution extends the discrete information propagation process inherent in the TextGCN to a diffusion process, thereby enabling the aggregation of information from larger neighborhoods. For each input graph, graph diffusion convolution manually fine-tunes the optimal neighborhood size for feature aggregation by conducting a parameter grid search on the validation set. However, this approach exhibits certain limitations and sensitivities in real-world applications. To obviate the need for a manual search for the optimal propagation neighborhood in Graph Diffusion Convolution, the Adaptive Diffusion Convolution strategy was introduced in TextGCN. This strategy supports the automatic learning of the optimal neighborhood from the data. During the message-passing process in each graph, all GCN layers and feature channels (dimensions) share the same neighborhood size. This strategy empowers GCNs to adapt more flexibly to various graph structures, thereby enhancing the model's generalization capability and performance. The

neighborhood radius can serve as a metric for quantifying the propagation distance of features across each layer. The neighborhood radius r is calculated as:

$$r = \frac{\sum_{s=0}^{\infty} \beta_s s}{\sum_{s=0}^{\infty} \beta_s}$$

where β_s denotes the influence of nodes at a s -step distance. A large r signifies that the model places a greater emphasis on distant nodes, thereby accentuating global information. Conversely, a small r indicates a focus on local information.

For graph convolutional networks (GCNs), when the neighborhood radius r equals 1, it only encompasses the range directly connected to the node. To access information beyond the direct connection range, it becomes necessary to stack multiple GCN layers to probe the high-order neighborhood. Models such as MixHop [41], JKNet [42], and SGC [43] strive to enhance the feature propagation function of the GCN by transitioning from a single-hop neighborhood to a multi-hop neighborhood. However, for all multi-hop models, the discrete nature of the number of hops renders the neighborhood radius r non-differentiable. This characteristic makes it improbable for r to adaptively participate in the Back-Propagation (BP) algorithm as a parameter. Consequently, the primary challenge in implementing the adaptive learning of the neighborhood radius lies in the non-differentiability of the radius.

Specially, we introduce Adaptive Diffusion Convolution into the construction process of the API call sequence TextGCN to capture the features of high-order text information more effectively. By employing this strategy to automatically learn the optimal neighborhood size strategy from the data, the TextGCN is fully integrated with the graph structure and all feature channels.

The emphasis is placed on the weight coefficients generated by the heat kernel version, denoted by $\beta_s = \frac{e^{-t} t^s}{s!}$. The heat kernel incorporates prior knowledge into the TextGCN model, implying that the feature propagation between nodes adheres to Newton's cooling law [44], i.e., the speed of feature propagation between two nodes is proportional to the difference in their features. Here, t can be interpreted as the diffusion time of node i .

Upon substituting the heat kernel formula into the generalized neighborhood radius Formula (9), and following a mathematical derivation involving exponential series, it can be deduced that the neighborhood radius r perfectly corresponds to the diffusion time t .

$$r_h = \frac{\sum_{s=0}^{\infty} \beta_s s}{\sum_{s=0}^{\infty} \beta_s} = \frac{\sum_{s=0}^{\infty} e^{-t} \frac{t^s}{s!} s}{\sum_{s=0}^{\infty} e^{-t} \frac{t^s}{s!}} = \frac{e^{-t} \sum_{s=0}^{\infty} \frac{t^s}{(s-1)!}}{e^{-t} \sum_{k=0}^{\infty} \frac{t^k}{k!}} = \frac{e^{-t} (e^t t)}{e^{-t} e^t} = t$$

This demonstrates that t , based on the heat kernel, is the neighborhood radius, implying that t becomes the perfect continuous substitute for the number of hops in the multi-hop model. This strategy provides a unique neighborhood size for each layer and channel, achieving more detailed adjustments and circumventing the need for manual adjustments.

4. Results

This section first presents the dataset employed in the experiment. Subsequently, we delve into a thorough analysis of the detailed steps involved in the implementation process, followed by a comprehensive evaluation of the experimental performance. Ultimately, we utilize these data for training and testing purposes to derive the experimental outcomes. Our code and dataset details can be found at www.sogithub.com/Yanggy/ABC (accessed on 18 March 2024).

4.1. Datasets

Our study utilized a comprehensive dataset consisting of 3000 ransomware samples and 2000 benign software samples carefully selected from reputable sources such as VirusShare, VirusTotal, and other well-known repositories. This selection aims to capture a wide range of ransomware behaviors and benign software patterns, ensuring the diversity

and representativeness of our dataset. In addition to our main dataset, we integrated data from the Ember dataset, malware_api_class dataset, and UNSW-NB15 dataset. These sources were integrated to expose our model to a wider range of malware features, behaviors, and attack types, including those that are not strictly classified as ransomware, thereby enhancing the robustness and accuracy of our detection model.

We used a Cuckoo Sandbox for the dynamic analysis of each sample, which enables us to capture the detailed execution of API call sequences. This analysis provides a rich feature set for distinguishing ransomware from benign software. We extracted relevant features from the API call sequence, with a focus on those features that have the highest frequency and significance in representing ransomware activities. Irrelevant information, including common stop words and uninformed API calls, was removed to increase the model's focus on important patterns. All features have been normalized to ensure scale consistency, thereby promoting more effective learning. To address the potential imbalance between ransomware and benign software samples and enhance the model's generalization ability, we adopted the Synthetic Minority Oversampling Technique (SMOTE). This technique generates composite ransomware samples by interpolating existing ones, enriching the diversity of ransomware behavior in our training set. When evaluating model performance, we focus on key indicators such as accuracy, precision, and recall to ensure that our research results can effectively identify ransomware and provide powerful tools for professionals in the field of network security.

4.2. Implementation Details

To accurately train the ADC-TextGCN model, our carefully constructed experimental environment mainly includes the following advanced hardware and software configurations: The core processor adopts the powerful i9-13900k, ensuring the efficiency and stability of the calculation process. All model training and testing work was completed on the top NVIDIA RTX 4090 GPU to achieve unparalleled processing speed and optimized performance. The development environment is based on the Windows 10 operating system and adopts the PyTorch framework and CUDA Toolkit 11.2. This choice significantly improves computational efficiency and optimizes the smoothness of program operation. In addition, the entire system runs on Ubuntu 22.04.4 LTS, further ensuring compatibility with advanced technology and system stability. This version is a Long Term Support release that receives regular updates, making it a reliable choice for maintaining consistent performance. This comprehensive configuration not only endows the experimental training environment with powerful computing power but also provides the optimal support and software environment for ADC-TextGCN model training, ensuring the efficient progress of research work and the output of excellent results.

The ADC-TextGCN model is structured to exploit the relational information inherent in textual data through graph-based learning. At its core, the model represents documents and words as nodes in a graph, where edges reflect co-occurrence relationships and document–word associations. This representation facilitates the learning of word and document embeddings in a unified feature space. The model includes two graph convolutional layers: the first layer converts the original feature representation into intermediate embeddings, and the second layer generates embeddings for final classification. The adaptive diffusion convolutional strategy is integrated into this architecture, allowing the model to adaptively adjust the neighborhood information aggregated at each node, thereby enhancing the model's ability to capture nuanced semantic relationships.

We set the learning rate (LR) to 0.01, as this was determined through preliminary experiments to balance convergence speed and training stability. Two graph convolutional layers are used to optimize the capturing of both local and global textual features without overfitting. Intermediate embeddings in the first layer are 200-dimensional, balancing representational capacity and computational efficiency. When applied to two layers with a dropout rate of 0.5, overfitting is prevented by randomly omitting a portion of the feature detectors at each training instance. To minimize the excessive complexity of model

weights, we used weight decay as a regularization method with a coefficient of 0.0005. The ADC-TextGCN is trained using a cross-entropy loss function, suitable for binary classification tasks like ransomware detection. Training is conducted over 300 epochs, with early stopping implemented to cease training if validation loss does not improve for 10 consecutive epochs. This strategy ensures the efficient use of computational resources while preventing overfitting. The Adam optimizer is selected for its adaptive learning rate properties, facilitating more efficient convergence. A grid search method is applied to explore the best combinations of the learning rate, dropout rate, and weight decay, ensuring optimal model performance. Our model settings are summarized in Table 2.

Table 2. The parameters used in our model and their respective values.

Parameters	Momentum	Weight Decay	Epoch	Batch Size	Initial Learning Rate
Our model	0.937	0.0005	300	32	0.01

4.3. Performance Estimation

The confusion matrix is a universally recognized method for assessing the performance of a model. It allows for the derivation of several key indicators, including the true positive rate (TPR), false positive rate (FPR), accuracy, Receiver Operating Characteristic (ROC), and Area Under the ROC Curve (AUC). In the specific context of ransomware sample detection, the AUC serves as a comprehensive metric of the model's detection capability, while accuracy validates the model's ability to correctly identify both benign software and ransomware samples. For this experiment, we selected TPR, FPR, AUC, and accuracy (ACC) as our evaluation metrics. A larger AUC value signifies a more effective classifier. The ROC is a two-dimensional curve, with FPR and TPR serving as its horizontal and vertical coordinates, respectively.

Within the confusion matrix, positive refers to a ransomware sample, while negative refers to a benign software sample. True positive indicates that the predicted result aligns with the actual situation, whereas false positive indicates a discrepancy between the prediction (ransomware) and the actual situation (benign software). Table 3 provides an overview of the advantages and disadvantages of each model, including ADC-TextGCN, which provides a detailed perspective on its relationship with other methods from the perspectives of effectiveness, complexity, and application scope.

Table 3. Comparative analysis of ransomware detection models.

Model/Approach	Advantages	Disadvantages
ADC-TextGCN	High accuracy and precision; effective integration of distant node information; adaptive neighborhood size, which enhances model's learning capability.	Requires significant computational resources for training; complexity in tuning for optimal neighborhood size.
CNN-based model	Excellent at capturing local and spatial dependencies; relatively simple to implement.	Struggles with long-range dependencies; less effective in capturing global context.
LSTM-based approach	Good at processing sequential data and capturing long-term dependencies.	Prone to vanishing gradient problem; computational inefficiency with long sequences.

This comparison table emphasizes the advanced capabilities of the ADC-TextGCN model to synthesize and learn extensive and complex ransomware data, which is attributed to its adaptive diffusion strategy. However, it also emphasizes the necessity for substantial computing power and the complex model optimization process, which may pose challenges in certain environments.

In contrast, while other models, such as CNNs and LSTM, offer valuable advantages, such as ease of implementation and effectiveness in processing sequential data, they all have limitations that ADC-TextGCN aims to overcome, especially in dealing with long-term dependencies and global context understanding.

4.4. COIR-PMI Experiment and α Factor Experiment

The co-occurrence retention algorithm, leveraging the COIR-PMI theory delineated in Section 3.2, was examined. Table 4 showcases the accuracy attained by various statistical methods when N equals 7. The table illustrates that the accuracy is enhanced when the weight between two word nodes is calculated using the COIR-PMI theory to preserve co-occurrence information. This approach bolsters the precision of the classification during training.

Table 4. COIR-PMI accuracy experiment.

Method	Accuracy (E)	Recall (E)	Accuracy (M)	Recall (M)	Accuracy (U)	Recall (U)
COIR-PMI	0.932	0.903	0.924	0.893	0.907	0.890
PMI	0.914	0.876	0.903	0.882	0.894	0.867

The letters in parentheses represent the first letter of the dataset, where E represents the Ember dataset, M represents the malware_api_class dataset, and U represents the UNSW-NB15 dataset. As the following table is identical to the previously discussed content; hence, a detailed explanation is not provided.

We investigated the influence of the α factor on the comprehensive performance of ADC-TextGCN. As depicted in Figure 7, our findings reveal that the accuracy initially escalates and subsequently declines with an increase in α , attaining a peak when α is set to 35.

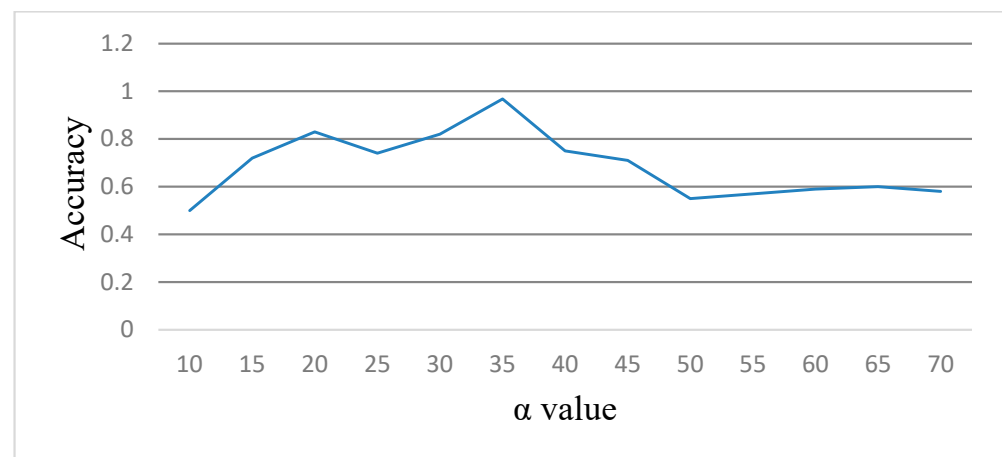


Figure 7. Results of α factor experiment.

4.5. Ablation Experiments

As depicted in Figure 8, we conducted a comparative analysis of various ransomware detection models, including the CNN-LSTM [45] hybrid, TextCNN [46], TextGCN [18], and LSTM [47] models. Initially, the CNN-LSTM hybrid model exhibits high accuracy, which, however, diminishes with an increase in the number of API call sequences. As the length of the API call sequences escalates, more sensitive API call functions emerge, thereby enhancing the model's detection accuracy. This trend persists until all sensitive API call functions are accounted for, at which point the network accuracy peaks at 0.966. Figure 8 illustrates that longer sequence lengths yield more precise results. Although the positive impact of increasing the sequence length eventually tapers off, ADC-TextGCN still outperforms the LSTM, CNN-LSTM hybrid, TextGCN, and TextCNN models in terms of accuracy.

The choice of these models for comparison is motivated by their status as classic models in the field of deep learning. Comparing ADC-TextGCN against these classic models provides a clear reference point for readers to understand the performance of our model. The comparison remains fair and meaningful, as all models were evaluated under the same conditions, even if the comparison models are not the latest. Furthermore, the choice of these widely available and easily understood models enhances the reproducibility of our experiments.

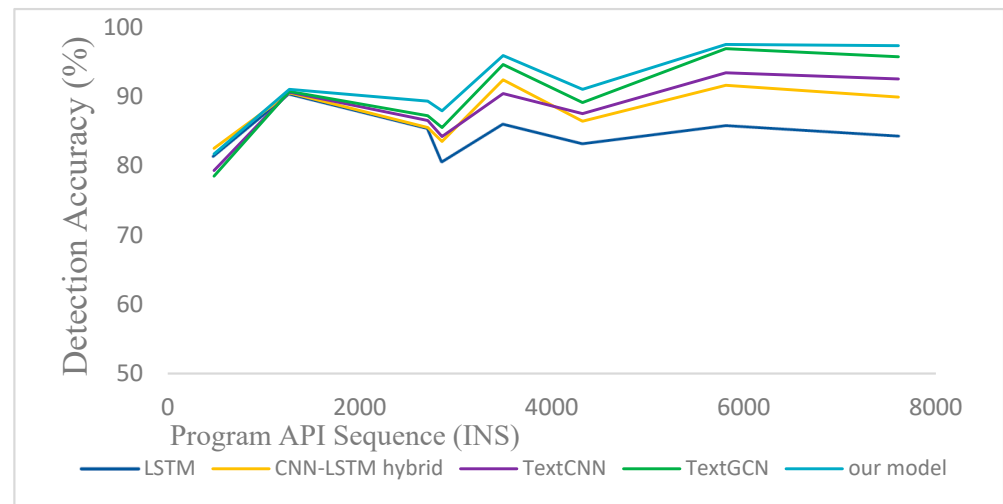


Figure 8. Comparison of different models.

The outcomes of the ablation experiment are delineated in Table 5. The results show a notable enhancement in accuracy with the combined application of the α factor and COIR-PMI, resulting in an accuracy increase of approximately 6.8% for dataset E compared to the base model without any enhancements. Furthermore, the addition of the diffusion radius t leads to further incremental improvements, achieving an impressive accuracy rate of 96.6% on dataset E. Similar trends are observed on datasets M and U, with accuracy improving to 95.3% and 92.7% respectively when all factors are applied. These results underscore the comprehensive effectiveness of the ADC-TextGCN, which significantly outperforms the baseline and other existing methods in accuracy.

Table 5. Ablation experiment.

α Factor	COIR-PMI	t (r)	Accuracy (E)	Recall (E)	Accuracy (M)	Recall (M)	Accuracy (U)	Recall (U)
×	×	×	0.875	0.723	0.861	0.763	0.814	0.761
✓	×	×	0.889	0.755	0.879	0.778	0.830	0.774
✓	✓	×	0.943	0.907	0.932	0.898	0.913	0.882
✓	✓	✓	0.966	0.914	0.953	0.903	0.927	0.902

Figure 9 shows the ablation studies under various diffusion settings. Graph Diffusion Convolution can be understood as fixing the heat kernel ' t ' with sparsification. The findings suggest that the efficacy of Graph Diffusion Convolution is partly due to its sparsification effect on the propagation matrix. Notably, if sparsification is removed, training the diffusion time parameter ' t ' for each feature channel and layer markedly outperforms merely fixing ' t ' to its initial value. By comparing the enhancements brought about by training ' t ' at different levels, Figure 9 also demonstrates that training ' t ' for each channel and layer makes the most contribution to the performance improvements in the node classification task.

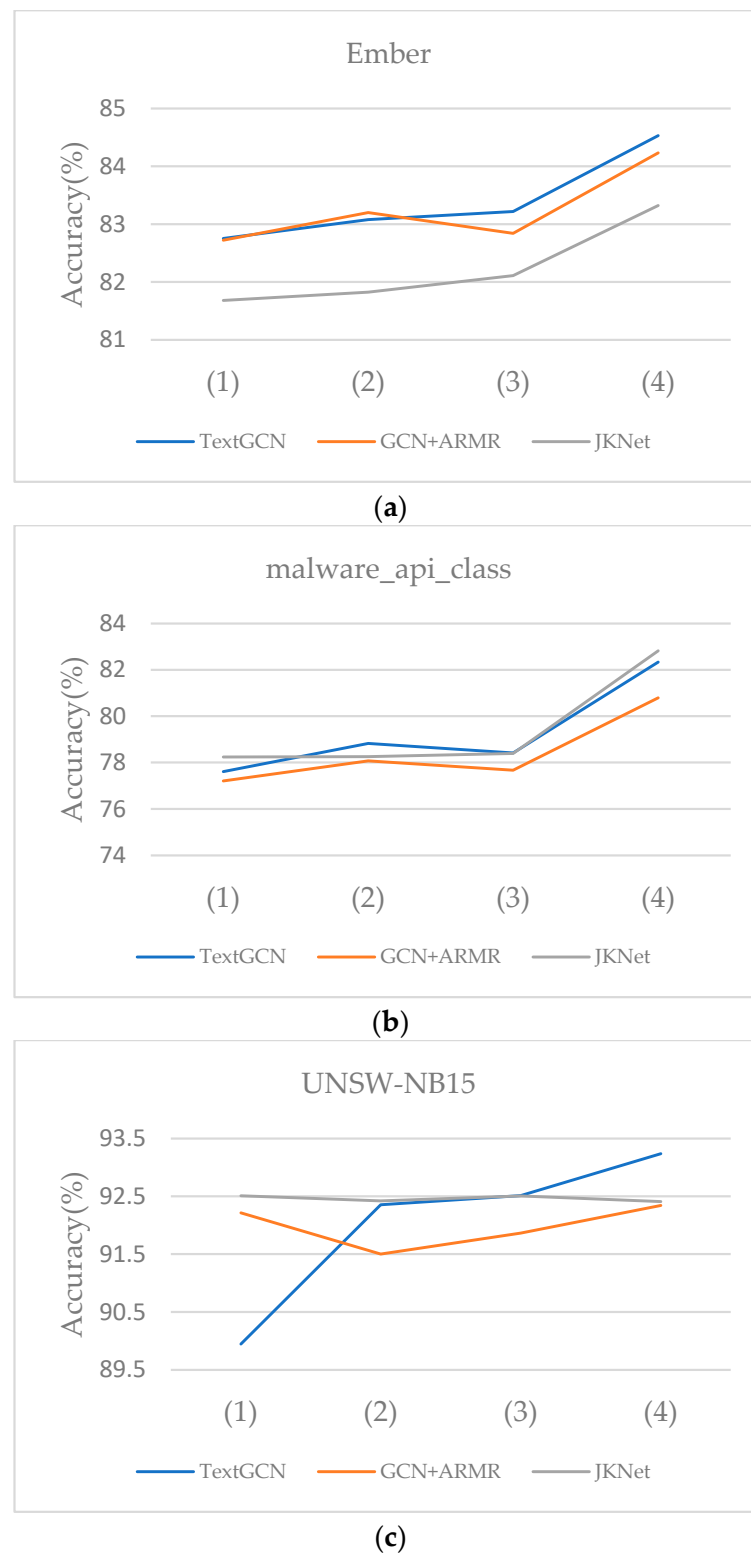


Figure 9. Ablation experiment: comparison of different training strategies. (1) Fixing ‘t’ to the initialization value. (2) Training a single ‘t’ for all layers. (3) Training a unique ‘t’ for each layer. and (4) Training a unique ‘t’ for each feature channel and layer. The experiments were conducted using three different datasets: (a) the Ember dataset, (b) the malware_api_class dataset, (c) and the UNSW-NB15 dataset.

We divided ablation experiments into four distinct scenarios. Among the scenarios, (1) can be viewed as Graph Diffusion Convolution without sparsification. Results from

these experiments show that training the diffusion time parameter directly from the data significantly enhances performance, especially when sparsification is removed.

Figure 10 presents the accuracy comparison between the traditional TextGCN methods and those enhanced with Graph Diffusion Convolution and Adaptive Diffusion Convolution strategies. It is evident from the figure that the Text Graph Convolutional Network utilizing the Graph Diffusion Convolution strategy significantly outperforms the traditional methods. Moreover, the ADC-TextGCN surpasses the version with the Graph Diffusion Convolution strategy applied. These results further validate the efficacy of our re-search findings and underscore the advantages of the ADC-TextGCN.

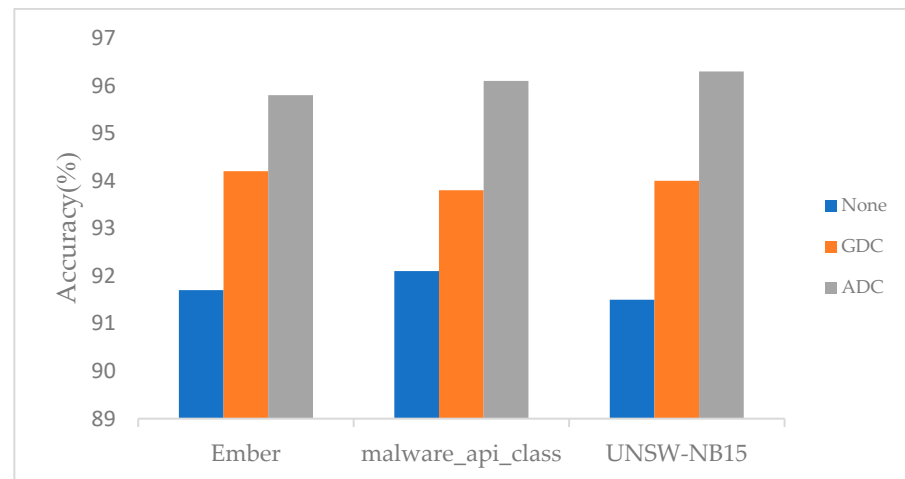


Figure 10. Comparing the accuracy of traditional TextGCN with versions enhanced by the GDC and ADC strategies.

To assess the impact of validation set size on model performance, a series of tests were conducted with various ratios between the training and validation sets while maintaining the same experimental conditions. The total number of nodes in the validation set remained unchanged, with only the distribution of nodes per class in the training set being adjusted. The outcomes of these tests are displayed in Figure 11. In most tested scenarios, ADC-TextGCN consistently outperforms both Graph Diffusion Convolution and the traditional TextGCN model. This indicates that the enhanced performance of ADC-TextGCN is not attributable to training additional parameters on a large validation set, but is inherent to its architectural design and learning approach.

The experiments were conducted on three different datasets: (a) the Ember dataset, (b) the Malware API Class dataset, and (c) the UNSW-NB15 dataset. Figure 11 aims to ascertain whether the observed improvement results from the overuse of the validation set, and the results indicate that this is not the case. Thus, we can conclude that the ADC-TextGCN model's superior performance is not due to overfitting on the validation set.

From the results in Figure 11, we can observe that (1) the ADC-TextGCN model demonstrates a clear advantage over the traditional TextGCN approach in terms of prediction accuracy. For example, in the UNSW-NB15 dataset, the ADC-TextGCN achieves an accuracy approximately 4.8% higher than the traditional TextGCN model. This suggests that the ADC-TextGCN is particularly effective in learning from global co-occurrence information. (2) The ADC-TextGCN also exhibits superior performance compared to the Graph Diffusion Convolution enhanced TextGCN, as evidenced by the higher accuracy rates across all datasets. In the Ember dataset, for instance, the ADC-TextGCN shows a nearly 1.6% increase in accuracy compared to the Graph Diffusion Convolution enhanced model, further underscoring the efficiency and effectiveness of the adaptive diffusion strategy employed by the ADC-TextGCN.

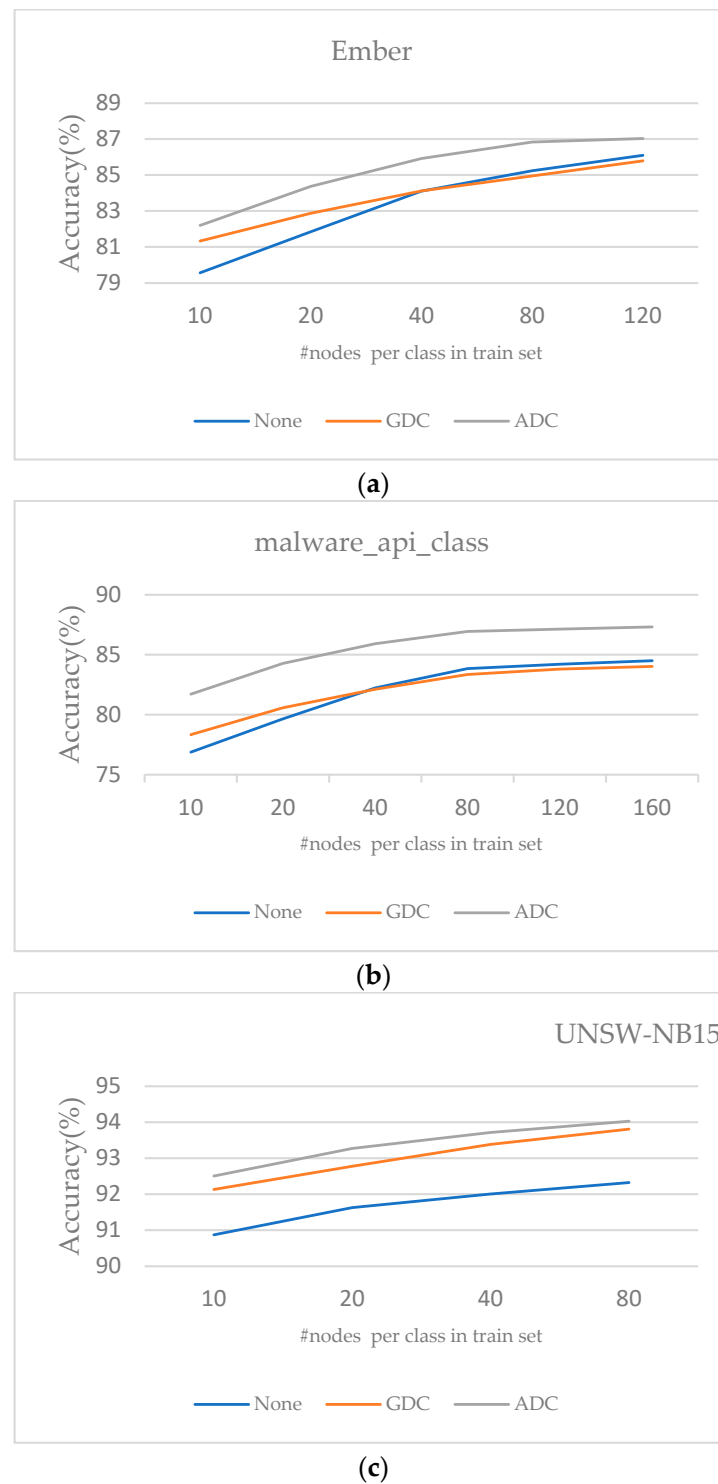


Figure 11. The influence of the number of training nodes: the X-axis denotes the number of nodes per class in the training set. ADC-TextGCN consistently outperforms Graph Diffusion Convolution and the traditional TextGCN model across most scenarios. The experiments were conducted using three different datasets: (a) the Ember dataset, (b) the malware_api_class dataset, (c) and the UNSW-NB15 dataset.

The excellent performance of the ADC-TextGCN model can be attributed to several key factors. The first is the understanding of the global context. Unlike traditional CNN and RNN models that mainly focus on local text features or sequence-based information, the ADC-TextGCN model utilizes global word co-occurrence and semantic relationships

between words in the entire dataset. This comprehensive perspective allows for a more detailed understanding of ransomware-related texts. Next is the Adaptive Diffusion Convolution strategy, which enables the model to dynamically adjust the diffusion process, enabling each node to effectively aggregate information from the optimal neighborhood size. This adaptability is particularly beneficial in capturing the complex patterns and behavioral characteristics of ransomware code, which may not be fully represented through fixed-size local windows or sequences. Finally, there is the efficiency of learning from structured data. Graph-based models are inherently adept at capturing irregularities and subtle structural differences in data, such as the interconnectivity of text in ransomware detection tasks. Compared to traditional models, this structural advantage facilitates more effective learning.

5. Discussion and Conclusions

By analyzing the behavior rules of ransomware, it has been established that long-distance discontinuous semantic information, word order information, and co-occurrence information are extremely essential for ransomware identification. Our ADC-TextGCN method retains comprehensive information by applying the sensitive API call sequences and the COIR-PMI. To address the issue that traditional TextGCN can only learn the node information of itself and its neighbors and to include more cross-layer node information in the process of multi-layer propagation training, we also incorporate an Adaptive Diffusion Convolution strategy, which supports the automatic learning of the optimal neighborhood from ransomware data and customizes the best propagation neighborhood size. Experimental results show that our model achieves an accuracy of 0.966. In the future, to further enhance the detection accuracy of this method, we will apply an attention mechanism to improve the ADC-TextGCN performance.

Although our research findings are promising, we acknowledge certain limitations, paving the way for future research. Firstly, the computational requirements of the ADC-TextGCN model may limit its deployment in resource-constrained environments. Further optimizing the architecture of the model and exploring more effective training algorithms can solve this problem. In addition, the evolutionary nature of ransomware attacks requires the continuous updating of the model's training data to ensure its relevance and effectiveness. Future research may also explore the integration of multimodal data sources, such as network traffic and user behavior logs, to enrich the learning environment of models and enhance their detection capabilities. Another challenge is that the model relies on a comprehensive and well-structured graphical representation of textual data. In situations where text data are sparse or lack clear semantic relationships, the performance of the model may be affected.

Our research holds significant practical value for cybersecurity professionals. The ADC-TextGCN model's efficient capability in detecting ransomware offers a robust tool against network threats, enabling more precise and effective identification and prevention of potential ransomware attacks. Given the increasingly complex digital environment, frequent ransomware attacks, and continuously evolving methods, the significance of this scientific advancement is particularly notable. It has established a solid defensive line for protecting information security. In our research, we emphasize two future research directions: improving model scalability and exploring the use of attention mechanisms, highlighting the tremendous potential for further progress in this area. As network security threats continue to evolve, so must our strategies and methods to counter these threats. Our research provides profound insights and powerful tools for cybersecurity professionals and researchers, laying a solid foundation for addressing future security challenges.

Author Contributions: Investigation, software, writing of original draft, validation, methodology, computing resources and automated data collection, G.Y.; conceptualization, methodology, supervision, and writing—review and editing, J.L.; data curation, Y.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Translational Application Project of the “Wise Eyes Action” (Project No. F2B6A194). We express our deepest gratitude to these organizations for their generous support.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets presented in this study are available at [www.github.com/Yanggy/ABC](https://github.com/Yanggy/ABC) (accessed on 18 March 2024).

Acknowledgments: We would like to express our gratitude to Yanhua Shao (The Sixth Research Institute of China Electronics and Information Industry Corporation) for providing computational resources and support. All authors have read and agreed to the published version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wilner, A.; Jeffery, A.; Lalor, J.; Matthews, K.; Robinson, K.; Rosolska, A.; Yorgoro, C. On the social science of ransomware: Technology, security, and society. *Comp. Strategy* **2019**, *38*, 347–370. [\[CrossRef\]](#)
2. Richardson, R.; North, M.M. Ransomware: Evolution, mitigation and prevention. *Int. Manag. Rev.* **2017**, *13*, 10.
3. Akbanov, M.; Vassilakis, V.G.; Logothetis, M.D. WannaCry ransomware: Analysis of infection, persistence, recovery prevention and propagation mechanisms. *J. Telecommun. Inf. Technol.* **2019**, *1*, 113–124. [\[CrossRef\]](#)
4. Mackenzie, P. WannaCry Aftershock. Sophos. 2019. Available online: <https://www.sophos.com/en-us/medialibrary/PDFs/technical-papers/WannaCry-Aftershock.pdf> (accessed on 6 March 2024).
5. Mattei, T.A. Privacy, confidentiality, and security of health care information: Lessons from the recent WannaCry cyberattack. *World Neurosurg.* **2017**, *104*, 972–974. [\[CrossRef\]](#)
6. Sun, Z.; Yin, L.; Li, C.; Zhang, W.; Li, A.; Tian, Z. The QoS and privacy trade-off of adversarial deep learning: An evolutionary game approach. *Comput. Secur.* **2020**, *96*, 101876. [\[CrossRef\]](#)
7. Potter, B.; Day, G. The effectiveness of anti-malware tools. *Comput. Fraud Secur.* **2009**, *2009*, 12–13. [\[CrossRef\]](#)
8. Gómez-Hernández, J.A.; Álvarez-González, L.; García-Teodoro, P. R-Locker: Thwarting ransomware action through a honeypot-based approach. *Comput. Secur.* **2018**, *73*, 389–398. [\[CrossRef\]](#)
9. Kok, S.H.; Abdullah, A.; Jhanjhi, N.Z.; Supramaniam, M. Prevention of crypto-ransomware using a pre-encryption detection algorithm. *Computers* **2019**, *8*, 79. [\[CrossRef\]](#)
10. Peng, W.; Li, F.; Zou, X.; Wu, J. Behavioral malware detection in delay tolerant networks. *IEEE Trans. Parallel Distrib. Syst.* **2013**, *25*, 53–63. [\[CrossRef\]](#)
11. Das, S.; Liu, Y.; Zhang, W.; Chandramohan, M. Semantics-based online malware detection: Towards efficient real-time protection against malware. *IEEE Trans. Inf. Forensics Secur.* **2015**, *11*, 289–302. [\[CrossRef\]](#)
12. Liu, K.; Xu, S.; Xu, G.; Zhang, M.; Sun, D.; Liu, H. A review of android malware detection approaches based on machine learning. *IEEE Access* **2020**, *8*, 124579–124607. [\[CrossRef\]](#)
13. Kim, J.Y.; Bu, S.J.; Cho, S.B. Zero-day malware detection using transferred generative adversarial networks based on deep autoencoders. *Inf. Sci.* **2018**, *460*, 83–102. [\[CrossRef\]](#)
14. Apruzzese, G.; Colajanni, M.; Ferretti, L.; Guido, A.; Marchetti, M. On the effectiveness of machine and deep learning for cyber security. In Proceedings of the 2018 10th International Conference on Cyber Conflict (CyCon), Tallinn, Estonia, 29 May–1 June 2018; pp. 371–390.
15. Al-Rimy, B.A.S.; Maarof, M.A.; Shaid, S.Z.M. Ransomware threat success factors, taxonomy, and countermeasures: A survey and research directions. *Comput. Secur.* **2018**, *74*, 144–166. [\[CrossRef\]](#)
16. Raff, E.; Barker, J.; Sylvester, J.; Brandon, R.; Catanzaro, B.; Nicholas, C.K. Malware detection by eating a whole exe. In Proceedings of the Workshops at the Thirty-Second AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, 2–7 February 2018.
17. Cao, D.; Zhang, X.; Ning, Z.; Zhao, J.; Xue, F.; Yang, Y. An efficient malicious code detection system based on convolutional neural networks. In Proceedings of the 2018 2nd International Conference on Computer Science and Artificial Intelligence, Shenzhen, China, 8–10 December 2018; pp. 86–89.
18. Yao, L.; Mao, C.; Luo, Y. Graph convolutional networks for text classification. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 27 January–1 February 2019; Volume 33, pp. 7370–7377.
19. Sgandurra, D.; Muñoz-González, L.; Mohsen, R.; Lupu, E.C. Automated dynamic analysis of ransomware: Benefits, limitations and use for detection. *arXiv* **2016**, arXiv:1609.03020.
20. Kharraz, A.; Robertson, W.; Balzarotti, D.; Bilge, L.; Kirda, E. Cutting the gordian knot: A look under the hood of ransomware attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 12th International Conference, DIMVA 2015, Milan, Italy, 9–10 July 2015*; Proceedings 2015; Springer International Publishing: Cham, Switzerland, 2015; pp. 3–24.

21. Chen, Q.; Bridges, R.A. Automated behavioral analysis of malware: A case study of wannacry ransomware. In Proceedings of the 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA), Cancun, Mexico, 18–21 December 2017; pp. 454–460.
22. Zhao, Z.; Wang, J.; Bai, J. Malware detection method based on the control-flow construct feature of software. *IET Inf. Secur.* **2014**, *8*, 18–24. [[CrossRef](#)]
23. Bae, S.I.; Lee, G.B.; Im, E.G. Ransomware detection using machine learning algorithms. *Concurr. Comput. Pract. Exp.* **2020**, *32*, e5422. [[CrossRef](#)]
24. Qin, B.; Wang, Y.; Ma, C. API call based ransomware dynamic detection approach using textCNN. In Proceedings of the 2020 International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE), Fuzhou, China, 12–14 June 2020; pp. 162–166.
25. Idrees, F.; Rajarajan, M.; Conti, M.; Chen, T.M.; Rahulamathavan, Y. PIndroid: A novel Android malware detection system using ensemble learning methods. *Comput. Secur.* **2017**, *68*, 36–46. [[CrossRef](#)]
26. Coronado-De-Alba, L.D.; Rodríguez-Mota, A.; Escamilla-Ambrosio, P.J. Feature selection and ensemble of classifiers for Android malware detection. In Proceedings of the 2016 8th IEEE Latin-American Conference on Communications (LATINCOM), Medellin, Colombia, 15–17 November 2016; pp. 1–6.
27. Gazet, A. Comparative analysis of various ransomware virii. *J. Comput. Virol.* **2010**, *6*, 77–90. [[CrossRef](#)]
28. Josse, S. White-box attack context cryptovirology. *J. Comput. Virol.* **2009**, *5*, 321–334. [[CrossRef](#)]
29. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Philip, S.Y. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2020**, *32*, 4–24. [[CrossRef](#)]
30. Kipf, T.N.; Welling, M. Semi-supervised classification with graph convolutional networks. *arXiv* **2016**, arXiv:1609.02907.
31. Defferrard, M.; Bresson, X.; Vandergheynst, P. Convolutional neural networks on graphs with fast localized spectral filtering. *Adv. Neural Inf. Process. Syst.* **2016**, *29*, 3844–3852.
32. Hamilton, W.; Ying, Z.; Leskovec, J. Inductive representation learning on large graphs. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 1024–1034.
33. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Lio, P.; Bengio, Y. Graph attention networks. *arXiv* **2017**, arXiv:1710.10903.
34. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How powerful are graph neural networks? *arXiv* **2018**, arXiv:1810.00826.
35. Gilmer, J.; Schoenholz, S.S.; Riley, P.F.; Vinyals, O.; Dahl, G.E. Neural message passing for quantum chemistry. In Proceedings of the International Conference on Machine Learning, Sydney, NSW, Australia, 6–11 August 2017; pp. 1263–1272.
36. Bruna, J.; Zaremba, W.; Szlam, A.; LeCun, Y. Spectral networks and locally connected networks on graphs. *arXiv* **2013**, arXiv:1312.6203.
37. Henaff, M.; Bruna, J.; LeCun, Y. Deep convolutional networks on graph-structured data. *arXiv* **2015**, arXiv:1506.05163.
38. Gasteiger, J.; Weißenberger, S.; Günnemann, S. Diffusion improves graph learning. *Adv. Neural Inf. Process. Syst.* **2019**, *32*, 13333–13354.
39. Zhao, J.; Dong, Y.; Ding, M.; Kharlamov, E.; Tang, J. Adaptive diffusion in graph neural networks. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 23321–23333.
40. Deng, Z.; Lloyd, H.; Xia, C.; Møller, A.P.; Liang, W.; Zhang, Y. Components of variation in female common cuckoo calls. *Behav. Process.* **2019**, *158*, 106–112. [[CrossRef](#)] [[PubMed](#)]
41. Abu-El-Haija, S.; Perozzi, B.; Kapoor, A.; Alipourfard, N.; Lerman, K.; Harutyunyan, H.; Ver Steeg, G.; Galstyan, A. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In Proceedings of the International Conference on Machine Learning, Long Beach, CA, USA, 10–15 June 2019; pp. 21–29.
42. Xu, K.; Li, C.; Tian, Y.; Sonobe, T.; Kawarabayashi, K.I.; Jegelka, S. Representation learning on graphs with jumping knowledge networks. In Proceedings of the International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018; pp. 5453–5462.
43. Wu, F.; Souza, A.; Zhang, T.; Fifty, C.; Yu, T.; Weinberger, K. Simplifying graph convolutional networks. In Proceedings of the International Conference on Machine Learning, Long Beach, CA, USA, 10–15 June 2019; pp. 6861–6871.
44. Widder, D.V. *The Heat Equation*; Academic Press: Cambridge, MA, USA, 1976.
45. She, X.; Zhang, D. Text classification based on hybrid CNN-LSTM hybrid model. In Proceedings of the 2018 11th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China, 8–9 December 2018; Volume 2, pp. 185–189.
46. Kim, Y. Convolutional neural networks for sentence classification. *arXiv* **2014**, arXiv:1408.5882.
47. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [[CrossRef](#)] [[PubMed](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.